

Proiect Informatică Aplicată II

Stație meteo Arduino

Mandan Andrei
IETTI/II/4LF633

Cuprins

Scopul proiectului	1
Componente Hardware:	
Arduino Mega	2
Arduino Nano	3
Senzor de temperatură și umiditate DHT22	4
Modul cu ceas în timp real DS3231	4
Modul comunicare wireless nRF24L01	5
Display TFT LCD	5
Alte componente	6
Schema logică	7
Diagrama de conectare	8
Biblioteci folosite	9
Probleme și rezolvări	9
Codul pentru placa Arduino Mega (partea de bază a proiectului).....	11
Codul pentru placa Arduino Nano (partea de transmitere la distanță).....	24
Bibliografie	26

Scopul proiectului

Am ales această temă deoarece mi s-a părut că ar fi un punct bun de începere pentru a învăța cum funcționează un Arduino cât și folosirea senzorilor pentru a realiza diverse proiecte. Arduino a fost cea mai bună alegere deoarece cunosc limbajul C++ și pentru că m-am mai jucat cu asemenea module prin școala generală, deși la un nivel mult mai mic.

Acest proiect are ca finalitate afișarea temperaturii și a umidității din 2 locuri diferite în același timp. Este folositor deoarece poți ști cu o acuratețe de 0.5 grade Celsius temperatura din casă cât și de afară (se poate plasa transmițătorul pe balcon), lucru foarte folositor mai ales iarna deoarece nu mai trebuie să cauți pe internet care este vremea de afară la momentul respectiv.

Componente Hardware:

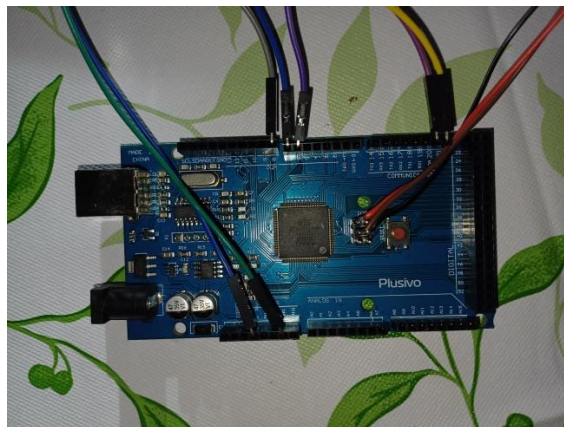
Arduino Mega

Proiectul are la bază un modul de dezvoltare de tip Arduino Mega. Este bazată pe pe microcontrolerul Atmega2560 și este utilizat pentru proiecte de electronică și programare. Este o alegere populară pentru aplicații complexe datorită numărului mare de pini și a capacității de memorie extinse.

Placa conține 54 de pini digitali, 16 analogici, are o memorie flash de 256 KB dintre care 8 KB rezervați pentru bootloader, 8KB SRAM și 4KB EEPROM; are 4 porturi UART, un conector I2C și SPI pentru conexiuni cu alte module sau senzori. Placa funcționează la o tensiune de 5V și are o ieșire de 5V și alta de 3.3V. Are o dimensiune de 5.3x10cm.

Modulul este potrivit pentru proiecte mai mari care necesită conexiuni hardware simultan. Suporta periferice complexe cum ar fi ecrane, motoare sau senzori multipli, fiind ideal pentru proiecte de robotica, automatizări sau dispozitie IoT.

În proiectul meu Arduino Mega-ul este placa principală, este cea care primește și afișează pe ecran toate datele de la senzorii conectați la ea cât și informațiile transmise de către Nano.



Arduino Nano

Placa Arduino Nano este o platformă de dezvoltare compactă, bazată pe microcontrolerul ATmega328 (sau ATmega168 în variante mai vechi), concepută pentru proiecte care necesită dimensiuni reduse și eficiență. Este una dintre cele mai populare alegeri pentru aplicații portabile și sisteme integrate.

Modulul conține 14 pini digitali (dintre care 6 pot fi folosiți ca PWM) și 8 analogici. Ca memorie are 32 KB flash (dintre care 2KB rezervați pentru bootloader), 2KB SRAM și 1 KB EEPROM. Placa lucrează la 5V, are un conector Mini-USB pentru alimentare și programare și are o dimensiune de 18 mm x 45 mm.

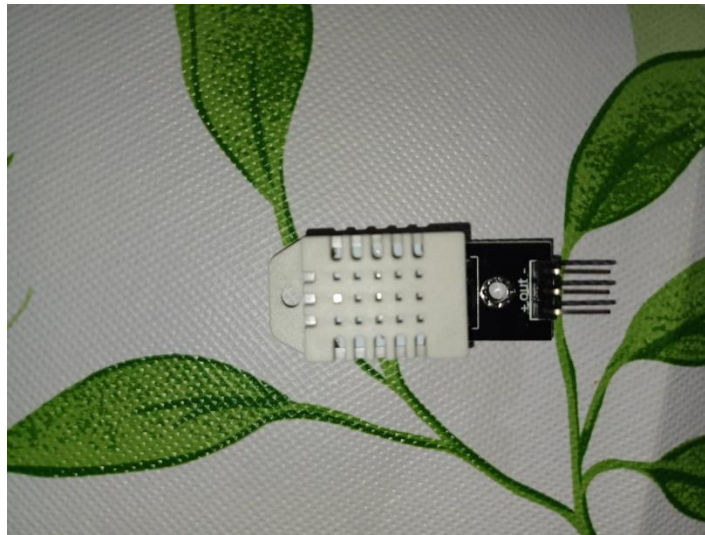
Nano-ul are în proiectul meu rolul de transmițător, deoarece el transmite prin intermediul unui transceiver wireless nRF24L01 către Mega-ul principal datele recepționate.



Senzor de temperatură și umiditate DHT22

Senzorul DHT22 este un senzor de umiditate și temperatură mic care folosește un senzor capacitiv de umiditate și un termistor pentru a măsura aerul din jur. Acesta oferă semnal digital pe pinul de date.

Senzorul are 3 pini: primul, marcat cu "+" se conectează la 5V sau 3.3V; al doilea, marcat cu "OUT" se conectează la un pin digital; al treilea, marcat cu "-" se conectează la un GND al plăcii de dezvoltare.



Modul cu ceas în timp real DS3231

Acest modul este un ceas în timp real extrem de precis, alimentat la 3.3V-5V. Scopul lui în proiect este de a ține minte ora în cazul în care proiectul nu este alimentat.



Modul comunicare wireless nRF24L01

Modulul are ca scop realizarea comunicației dintre părțile proiectului. El este un transceiver wireless bazat pe tehnologia radio frecvență. Funcționează pe banda de frecvență 2.4 GHz, are o viteză de transmisie de până la 2 mbps și, în condiții ideale, poate transmite până la 100 de metri în aer liber.



Display TFT LCD

Acest display este folosit pentru a afișa datele furnizate de senzori, cât și alte date relevante. Motivul pentru care am ales acest display a fost posibilitatea de a realiza o interfață grafică. El are o rezoluție de 480 x 320 pixeli, este bazat pe controllerul ILI9486 și este alimentat la 5V.



Alte componente

De asemenea aceste componente au trebuit interconectate. Pentru a face asta am folosit 2 breadboard-uri de 400 pini, cabluri mamă-tată, mamă-mamă și headeri de pini.

De precizat: Costul componentelor este de aproximativ **270 lei**, însă niște componente cum ar fi breadboard-urile, cabluri, și pinii îi aveam, iar placa Arduino Mega am primit-o de la o persoană apropiată cu ceva timp în urmă; așadar eu, real, am plătit aproape jumătate din valoarea proiectului din banii mei. Consider că deși prețul este mai mare, posibilitatea de a utiliza modulele în alte proiecte justifică prețul.

Schema logică

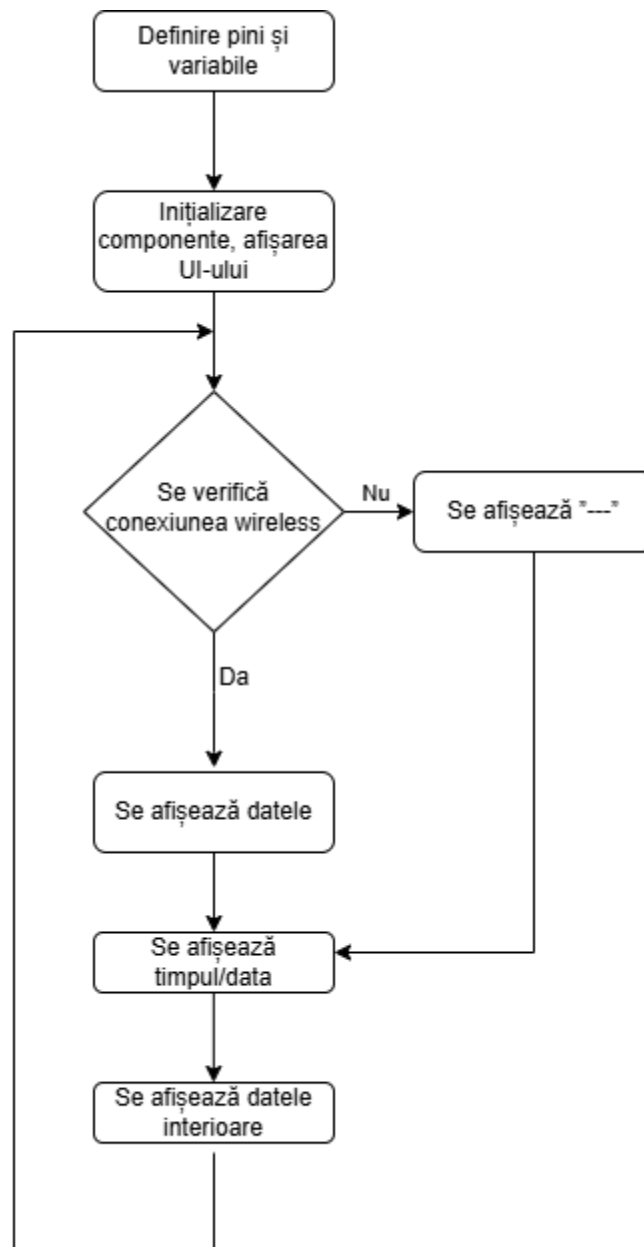
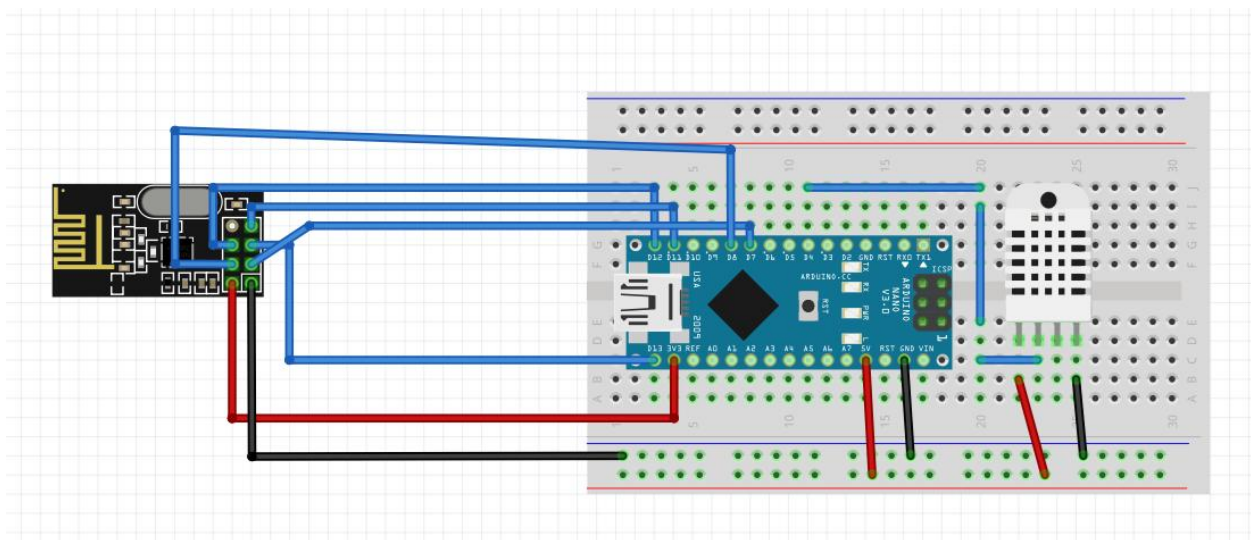
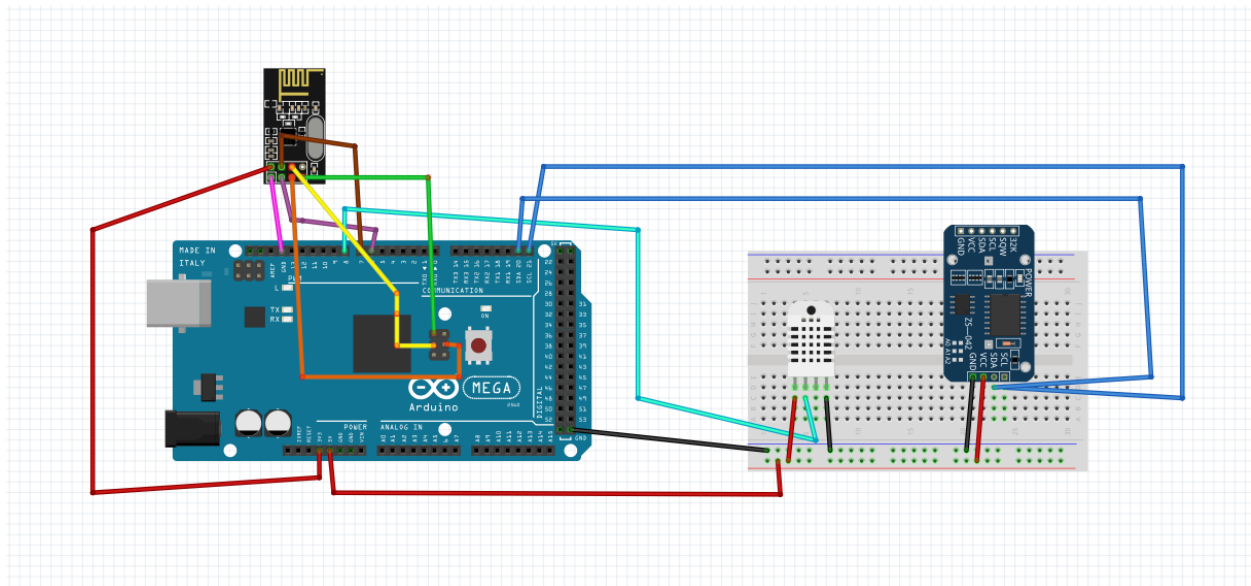


Diagrama de conectare

În programul Fritzing am realizat schema ambelor module.



Biblioteci folosite

În cadrul acestui proiect am folosit câteva biblioteci open-source pentru a putea folosi cu ușurință componentele. Acestea sunt:

- TFT_HX8357-master : pentru display
- RF24-master : pentru traneiverul wireless
- DHT-sensor-library-master : pentru senzorul de temperatură
- Sodaq_DS3231-master : pentru modulul Real-Time Clock

Probleme și rezolvări

Pe parcursul proiectului am întâmpinat mai multe probleme atât cu componentele cât și cu software-ul. Din cauza lor am pierdut un număr de ore dar consider că a meritat deoarece am învățat cum să caut mai bine și de unde să îmi iau informațiile.

Prima problemă pe care am întâmpinat-o a fost că nu puteam uploada cod pe Arduino Nano. Primeam o eroare cum că portul COM pe care era conectat la laptop era blocat. Am pierdut ore în șir citind pe forumuri și încercând fix-urile recomandate din comentarii: reinstalează drivere, dezactivează Serial Monitor-ul, dezactivează antivirusul și, desigur, dă-i restart la Windows. Am văzut aceleași recomandări pe mai multe site-uri însă nu mergea. Am ales să caut pe ChatGPT însă mi-a dat același chestii. I-am spus că tot nu merge și mi-a dat alte fix-uri doar că erau prea complexe și nu înțelegeam. Totuși unul din ele era să alegi din setările IDE-ului o versiune mai veche a microcontrollerului deoarece majoritatea plăcilor Arduino Nano aftermarket NU au cel mai recent microcontroller. Am schimbat din setări și a mers.

A doua problemă a fost cu display-ul. Biblioteca din descrierea produsului de pe Optimus Digital era pentru alt model decât cel specificat și nu am găsit nicăieri o altă bibliotecă. Nu mult a durat până am citit în comentarii că este greșit modelul specificat, deci biblioteca era bună (graba strică treaba).

Alte probleme au fost legate atât de display cât și de modulul nRF24L01. Modulul nu suportă 5V, așa că trebuia alimentat direct de la Mega din 3V3. Problema era că 2 pini care erau separați de restul pinilor displayului se suprapuneau exact pe pinul 3V3. Am găsit ulterior că acei 2 pini nu contează așa că i-am îndoit pentru a face loc. Altă problemă a fost conectarea pinilor SCK, MISO și MOSI la modul. Din cauza display-ului care vine fix pe Arduino, nu puteam conecta cei 2 pini direct cu cabluri jumper mamă-mamă la transceiver. Ei trebuiau lipiți cu cositor de cablu, lucru pe

care nu l-am mai făcut niciodată și pe care nu l-am putut face fără ajutor deoarece distanța dintre pini era foarte mică și nu am avut îndemânare. M-a ajutat tatăl meu la acest pas. De adăugat este că în timp ce mă chinuiam să învăț să folosesc ciocanul de lipit nu am văzut ca am ars niște cabluri și nu stiam de ce nu merg niște componente.

Ultima problemă, cea mai mare, a fost să pun cap la cap într-un program toate componentele. Deși mă descurc la C++, majoritatea cunoștințelor sunt din liceu așa că nu aveam conceptul bibliotecilor, a funcțiilor a obiectelor definite de alte persoane și a utilizării lor. Am parcurs funcțiile și metodele din biblioteci, am încercat să le înțeleg, iar prin acestea și cu ajutorul unor tutoriale de pe YouTube am reușit să iasă ceva. Important e că acest ceva funcționează cum trebuie.

Ca o încheiere, eu consider că acest proiect m-a ajutat enorm de mult în principal pentru a ieși din zona de confort, pentru a învăța inclusiv o aplicabilitate a programării (mereu mă întrebam „ok, scriu cod, dar cum îl pot implementa pe ceva fizic”) și cel mai important pentru a mă decide ce vreau să merg în continuare.

Codul pentru placa Arduino Mega (partea de bază a proiectului)

```
#include <TFT_HX8357.h>
#include <Sodaq_DS3231.h>
#include "DHT.h"
#include "RF24.h"

#define DHTPIN 8

#define DHTTYPE DHT22
DHT dht(DHTPIN, DHTTYPE);

RF24 myRadio (6, 7);
byte addresses[][6] = {"0"};

TFT_HX8357 tft = TFT_HX8357();

float remoteHumidity = 0.0;
float remoteTemperature = 0.0;

String dateString;
String hours;
int minuteNow=0;
int minutePrevious=0;

struct package
```

```
{  
  float temperature ;  
  float humidity ;  
};  
  
float previousIndoorHumidity = 0;  
float previousIndoorTemperature = 10;  
  
float previousRemoteHumidity = 0.1;  
float previousRemoteTemperature = 0.1;  
  
float indoorHumidity = 0;  
float indoorTemperature = 0;  
  
typedef struct package Package;  
Package data;  
  
void setup() {  
  
  Serial.begin(9600);  
  
  tft.init();  
  tft.setRotation(1);  
  tft.fillScreen(TFT_BLACK);  
  delay(100);  
}
```

```
rtc.begin();
dht.begin();
delay(2000);
//setRTCTime();

startWirelessCommunication();
printUI();
}

void loop() {

    checkForWirelessData();

    getAndPrintTime();

    printIndoorTemperature();
    printIndoorHumidity();

    printRemoteTemperature();
    printRemoteHumidity();
}

void setRTCTime()
{
```

```

    DateTime dt(2024, 11, 29, 22, 26, 30, 5);
    rtc.setDateTime(dt);
}

void startWirelessCommunication()
{
    myRadio.begin();
    myRadio.setChannel(115);
    myRadio.setPALevel(RF24_PA_MAX);
    myRadio.setDataRate( RF24_250KBPS );
    myRadio.openReadingPipe(1, addresses[0]);
    myRadio.startListening();
    delay(100);
}

void checkForWirelessData()
{
    if ( myRadio.available() )
    {
        while (myRadio.available())
        {
            myRadio.read( &data, sizeof(data) );
            previousRemoteTemperature = remoteTemperature;
            previousRemoteHumidity = remoteHumidity;
            remoteTemperature = data.temperature;
        }
    }
}

```



```

    remoteHumidity = data.humidity;
}
Serial.print("\nPackage:");
Serial.print("\n");
Serial.println(data.temperature);
Serial.println(data.humidity);
}
}

void printUI()
{

tft.drawRoundRect(5,5,470,71,5,TFT_WHITE);
tft.drawRoundRect(6,6,470,71,5,TFT_WHITE);

tft.drawRoundRect(5,90,220,225,5,TFT_WHITE);
tft.drawRoundRect(6,91,220,225,5,TFT_WHITE);

tft.drawRoundRect(250,90,220,225,5,TFT_WHITE);
tft.drawRoundRect(251,91,220,225,5,TFT_WHITE);

tft.fillRect(26,90,180,40,TFT_GREEN);
tft.fillRect(270,90,180,40,TFT_CYAN);

tft.setCursor(62,100);

```

```
tft.setTextColor(TFT_BLACK);  
tft.setTextSize(3);  
tft.print("TRANSM");
```

```
tft.setCursor(290,100);  
tft.setTextColor(TFT_BLACK);  
tft.setTextSize(3);  
tft.print("INTERIOR");
```

```
tft.setCursor(162,165);  
tft.setTextColor(TFT_GREEN);  
tft.setTextSize(6);  
tft.print("%");
```

```
tft.setCursor(412,165);  
tft.setTextColor(TFT_CYAN);  
tft.setTextSize(6);  
tft.print("%");
```

```
tft.setCursor(162,230);  
tft.setTextColor(TFT_GREEN);  
tft.setTextSize(6);  
tft.print("C");
```

```
tft.setCursor(145,230);
```

```
tft.setTextColor(TFT_GREEN);  
tft.setTextSize(2);  
tft.print("o");
```

```
tft.setCursor(412,230);  
tft.setTextColor(TFT_CYAN);  
tft.setTextSize(6);  
tft.print("C");
```

```
tft.setCursor(395,230);  
tft.setTextColor(TFT_CYAN);  
tft.setTextSize(2);  
tft.print("o");
```

```
}
```

```
void getAndPrintTime()
```

```
{
```

```
    delay(100);
```

```
    DateTime now = rtc.now();
```

```
    minuteNow = now.minute();
```

```
    if(minuteNow!=minutePrevious)
```

```
    {
```

```
        readSensor();
```

```

    dateString = getDayOfWeek(now.dayOfWeek())+" ";
    dateString = dateString+String(now.date())+"/"+String(now.month());
    dateString= dateString+"/"+String(now.year());
    minutePrevious = minuteNow;
    hours = String(now.hour());
    if(now.minute()<10)
    {
        hours = hours+":0"+String(now.minute());
    }else
    {
        hours = hours+": "+String(now.minute());
    }
    printTime();
}
}

```

```

void printTime()
{
    String dateAndTime = dateString+" "+hours;

    tft.setTextSize(2);
    char charBuf[25];
    dateAndTime.toCharArray(charBuf, 25);

    tft.setTextColor(TFT_WHITE, TFT_BLACK);

```

```
tft.drawCentreString(charBuf,240,25,2);  
}
```

```
String getDayOfWeek(int i)  
{  
  switch(i)  
  {  
    case 1: return "Luni";break;  
    case 2: return "Marti";break;  
    case 3: return "Miercuri";break;  
    case 4: return "Joi";break;  
    case 5: return "Vineri";break;  
    case 6: return "Sambata";break;  
    case 7: return "Duminica";break;  
    default: return "Luni";break;  
  }  
}
```

```
void readSensor()  
{  
  previousIndoorTemperature = indoorTemperature;  
  previousIndoorHumidity = indoorHumidity;  
  
  indoorHumidity = dht.readHumidity();  
  indoorTemperature = dht.readTemperature();  
}
```

```

    Serial.println(indoorTemperature);
    Serial.println(indoorHumidity);
}

void printIndoorTemperature()
{
    if(indoorTemperature != previousIndoorTemperature)
    {

        String temperature = String(indoorTemperature,1);

        tft.fillRect(270,232,120,40,TFT_BLACK);

        tft.setCursor(270,234);
        tft.setTextColor(TFT_CYAN);
        tft.setTextSize(5);
        tft.print(temperature);

        previousIndoorTemperature = indoorTemperature;
    }
}

void printRemoteHumidity()
{
    String humidity;

```

```

if(remoteHumidity != previousRemoteHumidity)
{
    if(remoteHumidity == 0.0 && remoteTemperature == 0.0)
    {
        humidity = "---";
    }else
    {
        humidity = String(remoteHumidity,1);
    }

    tft.fillRect(20,167,120,40,TFT_BLACK);

    tft.setCursor(20,167);
    tft.setTextColor(TFT_GREEN);
    tft.setTextSize(5);
    tft.print(humidity);

    previousRemoteHumidity = remoteHumidity;
}
}

void printRemoteTemperature()
{
    String temperature;
    if(remoteTemperature != previousRemoteTemperature)

```

```

{
  if(remoteHumidity == 0.0 && remoteTemperature == 0.0)
  {
    temperature = "---";
  }else
  {
    temperature = String(remoteTemperature,1);
  }

  tft.fillRect(20,232,120,40,TFT_BLACK);

  tft.setCursor(20,234);
  tft.setTextColor(TFT_GREEN);
  tft.setTextSize(5);
  tft.print(temperature);

  previousRemoteTemperature = remoteTemperature;
}
}

void printIndoorHumidity()
{
  if(indoorHumidity != previousIndoorHumidity)
  {

```



```
String humidity = String(indoorHumidity,1);
```

```
tft.fillRect(270,167,120,40,TFT_BLACK);
```

```
tft.setCursor(270,167);
```

```
tft.setTextColor(TFT_CYAN);
```

```
tft.setTextSize(5);
```

```
tft.print(humidity);
```

```
previousIndoorHumidity = indoorHumidity;
```

```
}
```

```
}
```

Codul pentru placa Arduino Nano (partea de transmitere la distanță)

```
#include "DHT.h"
```

```
#include <SPI.h>
```

```
#include "RF24.h"
```

```
#define DHTPIN 4
```

```
#define DHTTYPE DHT22
```

```
RF24 myRadio (7, 8);
```

```
byte addresses[][6] = {"0"};
```

```
const int led_pin = 13;
```

```
struct package
```

```
{
```

```
    float temperature ;
```

```
    float humidity ;
```

```
};
```

```
typedef struct package Package;
```

```
Package data;
```

```
DHT dht(DHTPIN, DHTTYPE);
```

```
void setup()
```

```
{
```

```
    Serial.begin(9600);
```

```

    pinMode(led_pin, OUTPUT);
    dht.begin();
    myRadio.begin();
    myRadio.setChannel(115);
    myRadio.setPALevel(RF24_PA_MAX);
    myRadio.setDataRate( RF24_250KBPS );
    myRadio.openWritingPipe( addresses[0]);
    delay(1000);
}

void loop()
{
    digitalWrite(led_pin, HIGH);
    readSensor();
    Serial.println(data.humidity);
    Serial.println(data.temperature);
    myRadio.write(&data, sizeof(data));
    digitalWrite(led_pin, LOW);
    delay(1000);
}

void readSensor()
{
    data.humidity = dht.readHumidity();
    data.temperature = dht.readTemperature();
}

```

Bibliografie

<https://app.diagrams.net/>

https://www.optimusdigital.ro/ro/compatibile-cu-arduino-mega/471-placa-de-dezvoltare-compatibila-cu-arduino-mega-2560-atmega2560-ch340.html?search_query=Placa+de+dezvoltare+compatibila+cu+Arduino+MEGA+2560+%28ATmega2560+++CH340%29&results=5

https://www.optimusdigital.ro/ro/compatibile-cu-arduino-nano/1686-placa-de-dezvoltare-compatibila-cu-arduino-nano-atmega328p-i-ch340.html?search_query=Placa+de+Dezvoltare+Compatibila+cu+Arduino+Nano+%28ATmega328p+%C8%99i+CH340%29a&results=6

https://www.optimusdigital.ro/ro/altele/12402-modul-cu-ceas-in-timp-real-ds3231.html?search_query=Modul+cu+Ceas+in+Timp+Real+DS3231+%28fara+baterie%29&results=1

https://www.optimusdigital.ro/ro/shield-uri-pentru-arduino/872-shield-lcd-pentru-arduino-mega-de-35-.html?search_query=Shield+LCD+pentru+Arduino+Mega+de+3.5%27%27&results=8

https://www.optimusdigital.ro/ro/senzori-senzori-de-temperatura/1449-modul-senzor-de-temperatura-i-umiditate-dht22.html?srsId=AfmBOorliM5x_SV3_2cp7J1ASdVBBYVHZwu6G_ntkQTQs7C6OvuduO6I

https://www.optimusdigital.ro/ro/ism-24-ghz/48-modul-tranceiver-nrf24l01-24-ghz.html?search_query=%09Modul+Transceiver+nRF24L01+%282.4+GHz%29&results=5

https://github.com/Bodmer/TFT_HX8357

<https://github.com/nRF24/RF24>

<https://github.com/adafruit/DHT-sensor-library>

https://github.com/SodaqMoja/Sodaq_DS3231